

12.2: Neo4j



UMD DATA605 - Big Data Systems

12.2: Neo4j

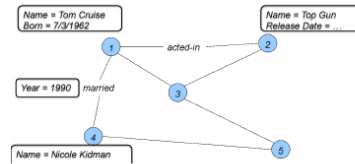
– **Instructor:** Dr. GP Saggese, gsaggese@umd.edu

1 / 16

2 / 16: Neo4j

Neo4j

- Graph DB storing data as Property Graph
 - Nodes, edges hold data as key-value pairs
- Graph structure enables flexible schema
 - Focus is on relationships between values
- Two querying languages
 - Cypher
 - Gremlin
- GUI or REST API
- Full ACID-compliant transactions
- High-availability clustering
- Incremental backups
- Run in small application or large server clusters



2 / 16

- **Graph DB storing data as Property Graph**
 - Neo4j is a type of database that uses a *property graph model*. This means data is stored in nodes and edges, with each holding data as key-value pairs. Nodes represent entities, while edges represent relationships between these entities.
- **Graph structure enables flexible schema**
 - The graph structure allows for a *flexible schema*, meaning it can easily adapt to changes in data structure. The focus is on the relationships between values, making it ideal for complex, interconnected data.
- **Two querying languages**
 - **Cypher**: A powerful, SQL-like language specifically designed for querying graphs.
 - **Gremlin**: Another language that can be used, though Cypher is more commonly associated with Neo4j.
- **GUI or REST API**
 - Neo4j can be accessed through a graphical user interface (GUI) or a REST API, providing flexibility in how users interact with the database.
- **Full ACID-compliant transactions**
 - Neo4j supports full ACID (Atomicity, Consistency, Isolation, Durability) transactions, ensuring reliable and secure data operations.
- **High-availability clustering**
 - It supports clustering for high availability, meaning it can handle large amounts of data and provide redundancy.
- **Incremental backups**
 - Neo4j offers incremental backups, allowing for efficient data recovery and management.
- **Run in small application or large server clusters**

- The database is versatile, capable of running on small applications or scaling up to large server clusters, making it suitable for various use cases.

3 / 16: Graph Data Model in Neo4j: Intuition

Graph Data Model in Neo4j: Intuition

- **Nodes**
 - Represent entities or objects
 - Connected via *relationships*
 - Have *properties* (key/value pairs)
- **Relationships**
 - Represent (directional) connections between nodes
 - Relationship types give semantic meaning to edges
 - Multiple relationships per node
 - Have *properties* (key/value pairs)
- **Properties**
 - Store key–value information on nodes and relationships
 - Named values (key is a string)
 - Indexed and constrained
 - Composite indexes from multiple properties
- **Labels**
 - Group nodes into sets with similar roles
 - Nodes may have multiple labels
 - Labels indexed for faster node retrieval
 - Native label indexes optimized for performance

3 / 16

- **Graph Data Model in Neo4j: Intuition**
- **Nodes**
 - Nodes are the fundamental units in a graph database, representing *entities* or *objects*. Think of them as the nouns in a sentence, like “Person” or “City”.
 - Nodes are connected to each other through *relationships*, which define how these entities interact or relate to one another.
 - Each node can have *properties*, which are essentially key/value pairs that store additional information about the node, such as a person’s name or age.
- **Relationships**
 - Relationships are the connections between nodes and are *directional*, meaning they have a start and an end point, like “Person A KNOWS Person B”.
 - The type of relationship provides *semantic meaning* to the connection, helping to understand the nature of the interaction between nodes.
 - A single node can have multiple relationships, allowing it to connect to various other nodes in different ways.
 - Like nodes, relationships can also have *properties* to store relevant information about the connection.
- **Properties**

- Properties are used to store detailed information on both nodes and relationships, using a key (a string) and a value.
- These properties can be *indexed* and *constrained* to improve query performance and ensure data integrity.
- You can create *composite indexes* using multiple properties, which helps in efficiently retrieving data based on complex queries.
- **Labels**
 - Labels are used to *group nodes* into sets that share similar characteristics or roles, such as all nodes representing “Employees”.
 - A node can have multiple labels, allowing it to belong to different groups or categories.
 - Labels are *indexed* to enable faster retrieval of nodes, making it easier to find all nodes with a specific label.
 - Neo4j uses *native label indexes* that are optimized for performance, ensuring quick access to nodes based on their labels.

4 / 16: Why Cypher is Powerful

Why Cypher is Powerful

- Direct mapping between query and graph structure
 - Encourages thinking in relationships
 - Reduces impedance mismatch with graph data
- Scales naturally with connected data
- Enables expressive exploratory queries

4 / 16

- **Why Cypher is Powerful**
- **Direct mapping between query and graph structure**
 - Cypher is a query language specifically designed for graph databases. This means that when you write a query in Cypher, it closely resembles the structure of the graph itself. This direct mapping makes it easier to understand and write queries because you are essentially describing the graph’s structure in your query.
 - This approach encourages you to think in terms of *relationships* rather than just indi-

vidual data points. In graph databases, relationships are first-class citizens, and Cypher helps you leverage this by making it intuitive to express these connections.

- By reducing the *impedance mismatch*—the disconnect between how data is stored and how it is queried—Cypher allows for more efficient and natural interaction with graph data.
- **Scales naturally with connected data**
 - Graph databases are designed to handle connected data efficiently. As your data grows and becomes more interconnected, Cypher queries can scale with it. This means that even as the complexity of your data increases, Cypher can still perform well, making it a robust choice for large datasets with many relationships.
- **Enables expressive exploratory queries**
 - Cypher allows for *expressive* queries, meaning you can write complex queries that explore the data in depth. This is particularly useful for exploratory data analysis, where you might not know exactly what you're looking for at the start. Cypher's syntax and capabilities make it easier to ask open-ended questions and discover insights within your data.

5 / 16: Basic Cypher Pattern Matching

Basic Cypher Pattern Matching

- Queries describe graph patterns to search for
 - Parentheses `()` represent nodes
 - Brackets `[]` represent relationships
 - Arrows `->` or `<-` show relationship direction

- Example

```
(a)-[:FRIEND_OF]->(b)
```

- Query Structure

```
MATCH [Nodes and relationships]
WHERE [Boolean filter statement]
RETURN [DISTINCT] [statements [AS alias]]
ORDER BY [Properties] [ASC or DESC]
SKIP [Number] LIMIT [Number]
```

5 / 16

- **Basic Cypher Pattern Matching**
 - Cypher is a query language used for interacting with graph databases, like Neo4j. It allows you to describe patterns in a graph to find specific data.
 - **Queries describe graph patterns to search for**
 - In Cypher, you use specific symbols to represent different parts of a graph:

- **Parentheses** `()` are used to denote *nodes*, which are the entities or objects in the graph.
- **Brackets** `[]` are used to denote *relationships*, which are the connections between nodes.
- **Arrows** `->` or `<-` indicate the *direction* of the relationship. This is important because relationships in graphs can be directional, meaning they have a start and an end point.
- **Example**
 - The example `(a)-[:FRIEND_OF]->(b)` shows a simple pattern where node `a` is connected to node `b` by a relationship labeled `FRIEND_OF`. The arrow indicates that `a` is a friend of `b`.
- **Query Structure**
 - Cypher queries have a specific structure to follow:
 - **MATCH** is used to specify the pattern of nodes and relationships you are looking for in the graph.
 - **WHERE** allows you to add conditions to filter the results based on certain criteria.
 - **RETURN** specifies what information you want to get back from the query. You can use **DISTINCT** to avoid duplicate results and **AS** to rename the returned data.
 - **ORDER BY** lets you sort the results based on properties of the nodes or relationships, either in ascending (ASC) or descending (DESC) order.
 - **SKIP** and **LIMIT** are used to paginate results, allowing you to skip a certain number of results and limit the number of results returned.

6 / 16: MATCH Clause

MATCH Clause

- Used to find patterns in the graph
 - Similar to `FROM ... WHERE` in relational databases
 - Does not modify data
 - Can match multiple patterns in one query
- Example

```
MATCH (p:Person)-[:LIVES_IN]->(c:City)
```

– **MATCH** Clause

- The **MATCH** clause is a fundamental part of querying in graph databases, particularly in systems like Neo4j. It is used to search for specific patterns within the graph data.
- Think of it as similar to the **FROM ... WHERE** clause in SQL used for relational databases. While SQL queries specify tables and conditions to filter rows, **MATCH** specifies nodes and relationships to find patterns.
- It's important to note that the **MATCH** clause is purely for querying and does not alter or modify the data in any way. It is used solely for retrieving data based on the specified patterns.
- One of the powerful features of the **MATCH** clause is its ability to match multiple patterns within a single query. This allows for complex queries that can retrieve interconnected data efficiently.

– **Example**

```
MATCH (p:Person)-[:LIVES_IN]->(c:City)
```

- In this example, the query is looking for a pattern where a node labeled **Person** is connected to a node labeled **City** through a relationship labeled **LIVES_IN**.
- The pattern **(p:Person)** represents a node with the label **Person**. The variable **p** is used to refer to this node in the query.
- The arrow **->** indicates the direction of the relationship, showing that the **Person** node is connected to the **City** node.
- This query will return all pairs of **Person** and **City** nodes that are connected by the **LIVES_IN** relationship.

7 / 16: Advanced Matching

Advanced Matching

- **RETURN** clause specifies what data to output
 - Can return nodes relationships or properties
 - Controls query result shape
 - Example


```
RETURN p.name, c.name
```
- Filtering with **WHERE**
 - Adds conditions to pattern matches
 - Works with properties labels and expressions
 - Often combined with **MATCH**
 - Example


```
WHERE p.age > 30
```
- Aggregation and Grouping
 - Uses functions like **count**, **avg**, **max**
 - Aggregation happens after **MATCH**
 - **GROUP BY** is implicit in **RETURN**
 - Example


```
RETURN c.name, count(p)
```

7 / 16

- **Advanced Matching**
 - The **RETURN** clause is crucial in a query as it determines what data will be displayed as the output. This means you can choose to return specific nodes, relationships, or properties from your data set. The shape of the query result is controlled by what you specify in the **RETURN** clause. For instance, if you want to see the names of two entities, you might use a command like `RETURN p.name, c.name`. This command will output the names of the entities represented by `p` and `c`.
- **Filtering with WHERE**
 - The **WHERE** clause is used to add conditions to your pattern matches, allowing you to filter the data based on specific criteria. It works with properties, labels, and expressions to refine the results of your query. Typically, **WHERE** is used in conjunction with the **MATCH** clause to specify which data should be included in the results. For example, `WHERE p.age > 30` filters the data to include only those entities where the age property is greater than 30.
- **Aggregation and Grouping**
 - Aggregation involves using functions like **count**, **avg**, and **max** to summarize data. This process occurs after the **MATCH** clause has been executed. In many query languages, **GROUP BY** is used to group data before aggregation, but in this context, it is implicit in the **RETURN** clause. For example, `RETURN c.name, count(p)` will group the data by `c.name` and count the number of occurrences of `p` for each group, providing a summary of the data.

8 / 16: Creating Data with CREATE

Creating Data with CREATE

- Used to add new nodes and relationships
- Pattern describes what should be created
- Executes exactly as written
- Example

```
CREATE
  (a:Person {name:"Alice"})
  -[:KNOWS]->
  (b:Person {name:"Bob"})
```

8 / 16

- **Creating Data with CREATE**
 - The **CREATE** command is a fundamental part of working with graph databases, such as Neo4j. It is used to add new elements to the database, specifically *nodes* and *relationships*.
 - **Used to add new nodes and relationships:** Nodes represent entities or objects, while relationships define how these nodes are connected. The **CREATE** command allows you to define both in one go.
 - **Pattern describes what should be created:** When using **CREATE**, you specify a pattern that outlines the structure of the nodes and relationships you want to add. This pattern is crucial because it dictates the exact configuration of the data being inserted.
 - **Executes exactly as written:** Unlike some other commands that might infer or adjust based on existing data, **CREATE** will execute precisely as you have written it. This means you need to be careful and precise in your syntax to avoid unintended data structures.
 - **Example:** The provided example demonstrates how to create two nodes labeled **Person**, with names “Alice” and “Bob”, and a relationship **KNOWS** between them. This is a straightforward way to represent that Alice knows Bob in the database.

9 / 16: Updating Graph Data

Updating Graph Data

- **SET** modifies properties or labels
- **REMOVE** deletes properties or labels
- Allows incremental graph evolution
- Example
`SET p.age = p.age + 1`

9 / 16

- **Updating Graph Data**
 - **SET modifies properties or labels:** In graph databases, nodes and relationships can have properties (like attributes in a table) and labels (which categorize nodes). The **SET** command is used to change these properties or add new labels to nodes or relationships. For example, if you have a node representing a person, you can update their age or add a new label to indicate a change in their status.
 - **REMOVE deletes properties or labels:** Just as you can add or modify properties and labels, you can also remove them using the **REMOVE** command. This is useful when a property is no longer relevant or a label no longer applies to a node or relationship.
 - **Allows incremental graph evolution:** Graph databases are dynamic, and the ability to update them incrementally is crucial. This means you can make small changes over time without needing to reload or recreate the entire graph. This flexibility is particularly important in applications where data changes frequently, such as social networks or recommendation systems.
 - **Example:** The example `SET p.age = p.age + 1` demonstrates how you can increment a property value. Here, `p` represents a node, and this command increases the `age` property by 1. This kind of operation is common in scenarios where you need to update data regularly, like tracking the age of users or the duration of an event.

10 / 16: Wine Suggestion Engine: Example 1/2

Wine Suggestion Engine: Example 1/2

- **Create a wine suggestion engine**
 - Wines categorized by:
 - Varieties (e.g., Chardonnay, Pinot Noir)
 - Regions (e.g., Bordeaux, Napa, Tuscany)
 - Vintage (year grapes harvested)
 - Track articles describing wines by authors
 - Users track favorite wines
 - ...

10 / 16

- **Wine Suggestion Engine: Example 1/2**
- **@Create a wine suggestion engine@**
 - **Wines categorized by:**
 - **Varieties (e.g., Chardonnay, Pinot Noir):** This refers to the different types of grapes used to make wine. Each variety has unique characteristics that affect the wine's flavor, aroma, and texture. Understanding these can help in suggesting wines that match user preferences.
 - **Regions (e.g., Bordeaux, Napa, Tuscany):** Wines are often associated with the regions they come from, as climate, soil, and local winemaking traditions influence their taste. By categorizing wines by region, the engine can recommend wines that reflect specific regional qualities.
 - **Vintage (year grapes harvested):** The year the grapes were harvested can significantly impact the wine's taste due to varying weather conditions each year. Some vintages are considered better than others, and this information can be crucial for making informed suggestions.
 - **Track articles describing wines by authors:** This involves collecting and analyzing wine reviews and articles written by experts. These articles can provide insights into wine quality and trends, which can be used to enhance the suggestion engine's recommendations.
 - **Users track favorite wines:** Allowing users to keep a record of their favorite wines helps the engine learn their preferences over time. This data can be used to personalize suggestions, making them more relevant and appealing to each user.

11 / 16: Wine Suggestion Engine: Example 2/2

Wine Suggestion Engine: Example 2/2

– **Relational approach**

– Create various tables

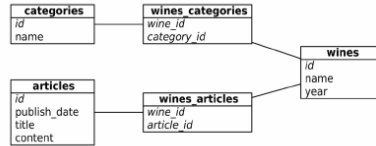
- **wines**: (id, name, year)
- **wines_categories** (wine_id, category_id)
- **category** table (id, name)
- **wines_articles** (wine_id, article_id)
- **articles** (id, publish_date, title, content)

– Relationships are

- **produced**
- **reported_on**
- **grape_type**

– **Problem with relational approach**

- There isn't much of a schema
- Lots of incomplete data
- An old saying in relational DB world: *"On a long enough timeline all fields become optional"*



11 / 16

– **Relational approach**

- The slide discusses a relational database model for a wine suggestion engine.

– **Tables:**

- **wines**: Contains basic information about wines, such as **id**, **name**, and **year**.
- **wines_categories**: Links wines to their categories using **wine_id** and **category_id**.
- **category**: Lists categories with **id** and **name**.
- **wines_articles**: Connects wines to articles using **wine_id** and **article_id**.
- **articles**: Contains article details like **id**, **publish_date**, **title**, and **content**.

– **Relationships:**

- *Produced*: Likely refers to the production details of the wine.
- *Reported on*: Indicates articles that discuss specific wines.
- *Grape type*: Could relate to the category or type of grape used in the wine.

– **Problem with relational approach**

- **Schema limitations**: The slide points out that the schema might not be well-defined or comprehensive.
- **Incomplete data**: There is a challenge with missing or optional fields, which can lead to incomplete datasets.
- **Old saying**: Highlights a common issue in relational databases where fields can become optional over time, leading to inconsistencies.

12 / 16: Cypher Example

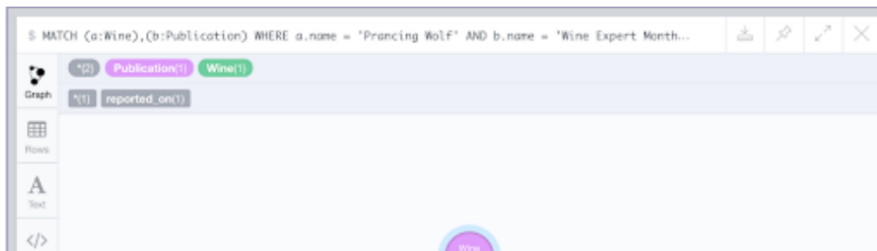
Cypher Example

- **Graph DB approach:** provide values and structure only where necessary

```
CREATE (w:Wine
      {name: "Prancing Wolf",
       style: "ice wine",
       vintage: 2015})

CREATE (p:Publication
      {name: "Wine Expert Monthly"})

MATCH (p:Publication
      {name: "Wine Expert Monthly"}),
      (w:Wine {name: "Prancing Wolf",
               vintage: 2015})
CREATE (p)-[r:reported_on]->(w)
```



- **Graph DB approach:** This approach focuses on providing values and structure only where necessary. It allows for efficient data modeling and querying by emphasizing relationships between data points rather than storing data in a rigid table format.
- **Cypher Query Explanation:**
 - **CREATE (w:Wine {name: “Prancing Wolf”, style: “ice wine”, vintage: 2015})**: This command creates a node labeled `Wine` with properties such as `name`, `style`, and `vintage`. Here, a specific wine called “Prancing Wolf” is being added to the graph database.
 - **CREATE (p:Publication {name: “Wine Expert Monthly”})**: This command creates another node labeled `Publication` with a `name` property. It represents a publication named “Wine Expert Monthly”.
 - **MATCH and CREATE Relationship:**
 - **MATCH (p:Publication {name: “Wine Expert Monthly”}), (w:Wine {name: “Prancing Wolf”, vintage: 2015})**: This part of the query finds the previously created nodes for the publication and the wine.
 - **CREATE (p)-[r:reported_on]->(w)**: This establishes a relationship labeled `reported_on` from the publication node to the wine node, indicating that the publication has reported on the wine.
- **Graph Visualization:** The image shows a simple graph with two nodes connected by a relationship. The nodes represent the publication and the wine, and the directed edge shows the reporting relationship.
- **Key Takeaway:** This example demonstrates how graph databases like Neo4j use nodes

and relationships to model complex data structures, allowing for intuitive and flexible data representation.

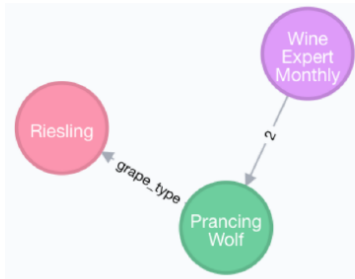
13 / 16: Cypher Example

Cypher Example

```
MATCH (p:Publication {name: "Wine Expert Monthly"}),
      (w:Wine {name: "Prancing Wolf"})
CREATE (p)-[r:reported_on {rating: 2}]->(w)
```

```
CREATE (g:GrapeType {name: "Riesling"})
```

```
MATCH (w:Wine {name: "Prancing Wolf"}),
      (g:GrapeType {name: "Riesling"})
CREATE (w)-[r:grape_type]->(g)
```



13 / 16

- **Cypher Query Explanation**
 - The Cypher query is used to interact with a graph database, specifically Neo4j.
 - **MATCH Clause:** This part of the query searches for existing nodes.
 - It looks for a **Publication** node with the name “Wine Expert Monthly”.
 - It also searches for a **Wine** node named “Prancing Wolf”.
 - **CREATE Clause:** This creates relationships between nodes.
 - A **reported_on** relationship is created from the **Publication** node to the **Wine** node, with a property **rating** set to 2.
 - A new node **GrapeType** with the name “Riesling” is created.
 - Another relationship **grape_type** is established from the **Wine** node to the **GrapeType** node.
- **Graph Visualization**
 - The diagram visually represents the nodes and relationships.
 - **Nodes:** Represent entities such as “Wine Expert Monthly”, “Prancing Wolf”, and “Riesling”.
 - **Relationships:** Arrows indicate connections between nodes.
 - The arrow from “Wine Expert Monthly” to “Prancing Wolf” shows the **reported_on** relationship with a rating of 2.
 - The arrow from “Prancing Wolf” to “Riesling” indicates the **grape_type** relationship.
- **Context and Importance**

- This example demonstrates how to model relationships in a graph database.
- It highlights the flexibility of graph databases in representing complex relationships.
- Understanding these concepts is crucial for efficiently querying and managing interconnected data.

14 / 16: Cypher Example

Cypher Example

```
CREATE (wr:Winery {name: "Prancing Wolf Winery"})
MATCH (w:Wine {name: "Prancing Wolf"}),
      (wr:Winery {name: "Prancing Wolf Winery"})
CREATE (wr)-[r:produced]->(w)
CREATE (w:Wine
      {name:"Prancing Wolf", style: "Kabinett", vintage: 2002})
CREATE (w:Wine
      {name: "Prancing Wolf", style: "Spätlese", vintage: 2010})
MATCH (wr:Winery
      {name: "Prancing Wolf Winery"}),(w:Wine {name: "Prancing Wolf"})
CREATE (wr)-[r:produced]->(w)
MATCH (w:Wine), (g:GrapeType {name: "Riesling"})
CREATE (w)-[r:grape_type]->(g)
```



14 / 16

- **Cypher Example:** This slide provides an example of using Cypher, a query language for Neo4j, which is a graph database. The example demonstrates how to create nodes and relationships between them.
- **Creating Nodes:**
 - The **CREATE** command is used to add nodes to the graph. Here, a node for a winery named “Prancing Wolf Winery” is created.
 - Additional nodes for wines named “Prancing Wolf” with different styles and vintages are also created.
- **Matching Nodes:**
 - The **MATCH** command is used to find existing nodes. It looks for a wine node and a winery node with specific names.
- **Creating Relationships:**
 - The **CREATE** command is also used to establish relationships between nodes. For example, the winery node is connected to the wine nodes with a **produced** relationship.
 - Another relationship is created between wine nodes and a grape type node named “Riesling”.

– Graph Visualization:

- The diagram illustrates the relationships between the winery, wines, and grape type. It shows how the “Prancing Wolf Winery” produces different wines, and how these wines are associated with the “Riesling” grape type.
- The visualization helps in understanding the interconnected nature of the data in a graph database.

15 / 16: Cypher Example

Cypher Example

- Add a social component to the wine graph
 - People preference for wine
 - Relationships with one another
- The changes were made “superimposing” new relationships without changing the previous data ::::: {.column width=50%}

```
CREATE (p:Person {name: "Alice"})
```

```
MATCH (p:Person {name: "Alice"}),
      (w:Wine {name: "Prancing Wolf",
               style: "ice wine"})
CREATE (p)-[r:likes]->(w)
```

```
CREATE (p:Person {name: "Patty"})
```

```
MATCH (p1:Person {name: "Patty"}),
      (p2:Person {name: "Tom"})
CREATE (p1)-[r:friends]->(p2)
```

```
::: ::: {.column width=45%}
```

4: Prancing Wolf Kabinett 2002

15 / 16

- **Add a social component to the wine graph**
 - The slide demonstrates how to enhance a wine graph by incorporating social elements. This involves adding nodes and relationships that represent people’s preferences and their connections with each other.
 - **People preference for wine:** This is shown by creating relationships between people and the wines they like. For example, Alice likes “Prancing Wolf” ice wine.
 - **Relationships with one another:** Social connections are depicted by creating relationships like “friends” between people, such as between Patty and Tom.
- **The changes were made “superimposing” new relationships without changing the previous data**
 - This means that the new social relationships are added on top of the existing wine data without altering the original structure. This approach allows for the enrichment of the graph with additional layers of information.
- **Cypher Code Explanation**
 - The Cypher code snippet shows how to create nodes and relationships in a graph database.
 - `CREATE (p:Person {name: "Alice"})`: This line creates a new person node named

Alice.

- MATCH and CREATE statements are used to establish relationships, such as Alice liking a specific wine and Patty being friends with Tom.
- **Graph Visualization**
 - The accompanying image visually represents the nodes and relationships. It shows how people are connected to each other and to the wines they like, illustrating the social component added to the graph.

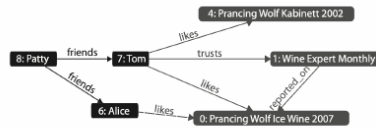
16 / 16: Cypher: Query Example

Cypher: Query Example

```
MATCH (p:Person
{name: "Alice"})-->(n)
RETURN n;
```

```
MATCH (p:Person
{name: "Alice"})-->(other: Person)
RETURN other.name;
```

```
MATCH (fof:Person)-[:friends]-(f:Person)-[:friends]-(p:Person {name: "Alice"})
RETURN fof.name;
```



16 / 16

- **MATCH (p:Person {name: “Alice”})-->(n) RETURN n;**
 - This query finds all nodes directly connected to the node labeled “Alice” and returns them.
 - In the context of the graph, it identifies all entities Alice is directly related to, such as items she likes or people she knows.
 - The arrow --> indicates a directed relationship from Alice to another node.
- **MATCH (p:Person {name: “Alice”})-->(other: Person) RETURN other.name;**
 - This query specifically looks for other people connected to Alice and returns their names.
 - It filters the results to only include nodes labeled as **Person**.
 - Useful for identifying Alice’s direct social connections.
- **MATCH (fof:Person)-[:friends]-(f:Person)-[:friends]-(p:Person {name: “Alice”}) RETURN fof.name;**
 - This query finds friends of Alice’s friends, often referred to as “friends of friends.”
 - It returns the names of these indirect connections, expanding the social network view.
 - The **[:friends]** specifies the type of relationship being queried, ensuring only friendship links are considered.